

.Encoding of a Turing Machine

Example 9.1: Let the TM in question be

$$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$$

where δ consists of the rules:

$$\delta(q_1, 1) = (q_3, 0, R)$$

$$\delta(q_3, 0) = (q_1, 1, R)$$

$$\delta(q_3, 1) = (q_2, 0, R)$$

$$\delta(q_3, B) = (q_3, 1, L)$$

The codes for each of these rules, respectively, are:

0100100010100

0001010100100

00010010010100

0001000100010010

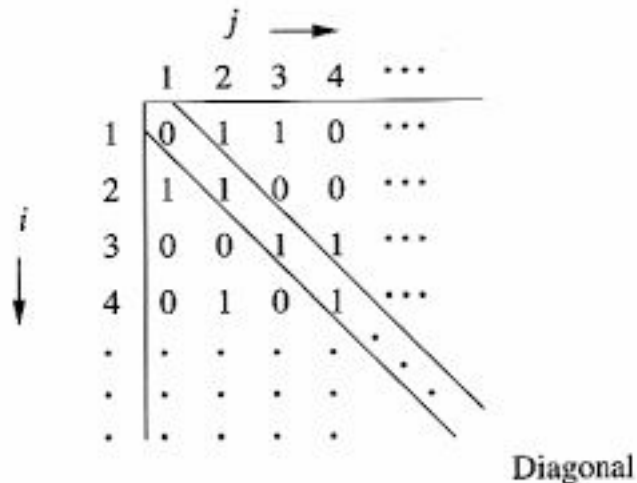
.Diagonalization Language

Q. Prove that real numbers are uncountable infinite sets

Q. Prove that rational and natural numbers are countable infinite sets

Q. A language L is recursively enumerable iff there exists a TM that enumerates all strings of L .

- The language L_d , the *diagonalization language*, is the set of strings w_i such that w_i is not in $L(M_i)$.



.real numbers are uncountable infinite sets

1. **Assume for contradiction** that the real numbers in $(0, 1)$ are countable. Then, there exists a bijection $f : \mathbb{N} \rightarrow (0, 1)$, meaning we can list all real numbers in $(0, 1)$ as a sequence:

$$f(1), f(2), f(3), \dots$$

Each $f(n)$ can be written as an infinite decimal expansion:

$$f(1) = 0.a_{11}a_{12}a_{13} \dots$$

$$f(2) = 0.a_{21}a_{22}a_{23} \dots$$

$$f(3) = 0.a_{31}a_{32}a_{33} \dots$$

$$\vdots$$

where each a_{ij} is a digit from 0 to 9.

2. **Construct a new real number** $x = 0.b_1b_2b_3 \dots$ in $(0, 1)$ by defining its digits as follows:

$$b_n = \begin{cases} 2 & \text{if } a_{nn} \neq 2, \\ 3 & \text{if } a_{nn} = 2. \end{cases}$$

(The choice of 2 and 3 is arbitrary; the key is that $b_n \neq a_{nn}$.)

3. **Observe that x differs from every $f(n)$** in the list:

- $x \neq f(1)$ because $b_1 \neq a_{11}$.
- $x \neq f(2)$ because $b_2 \neq a_{22}$.
- $x \neq f(3)$ because $b_3 \neq a_{33}$.
- And so on for every n .

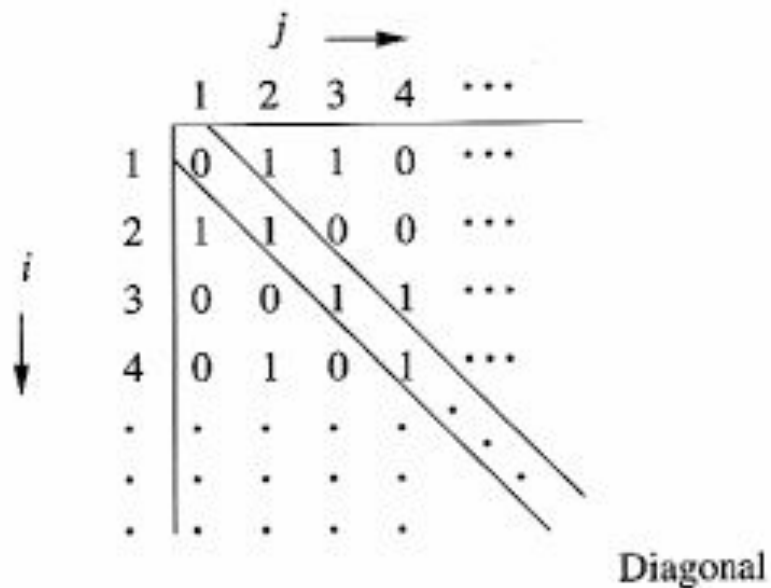
4. **Contradiction:** Since $x \in (0, 1)$ but x is not in the list $f(1), f(2), f(3), \dots$, the function f cannot be surjective. This contradicts our assumption that f is a bijection.

.rational and natural numbers are countable infinite sets

	1	2	3	4	5	6	7	8	...
1	$\frac{1}{1}$	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$	$\frac{1}{7}$	$\frac{1}{8}$	...
2	$\frac{2}{1}$	$\frac{2}{2}$	$\frac{2}{3}$	$\frac{2}{4}$	$\frac{2}{5}$	$\frac{2}{6}$	$\frac{2}{7}$	$\frac{2}{8}$	...
3	$\frac{3}{1}$	$\frac{3}{2}$	$\frac{3}{3}$	$\frac{3}{4}$	$\frac{3}{5}$	$\frac{3}{6}$	$\frac{3}{7}$	$\frac{3}{8}$	...
4	$\frac{4}{1}$	$\frac{4}{2}$	$\frac{4}{3}$	$\frac{4}{4}$	$\frac{4}{5}$	$\frac{4}{6}$	$\frac{4}{7}$	$\frac{4}{8}$	...
5	$\frac{5}{1}$	$\frac{5}{2}$	$\frac{5}{3}$	$\frac{5}{4}$	$\frac{5}{5}$	$\frac{5}{6}$	$\frac{5}{7}$	$\frac{5}{8}$	...
6	$\frac{6}{1}$	$\frac{6}{2}$	$\frac{6}{3}$	$\frac{6}{4}$	$\frac{6}{5}$	$\frac{6}{6}$	$\frac{6}{7}$	$\frac{6}{8}$	...
7	$\frac{7}{1}$	$\frac{7}{2}$	$\frac{7}{3}$	$\frac{7}{4}$	$\frac{7}{5}$	$\frac{7}{6}$	$\frac{7}{7}$	$\frac{7}{8}$	...
8	$\frac{8}{1}$	$\frac{8}{2}$	$\frac{8}{3}$	$\frac{8}{4}$	$\frac{8}{5}$	$\frac{8}{6}$	$\frac{8}{7}$	$\frac{8}{8}$	...
⋮	⋮								

.Prove that L_d is Non-R.E.

- The language L_d , the *diagonalization language*, is the set of strings w_i such that w_i is not in $L(M_i)$.



9.1.4 Proof That L_d Is Not Recursively Enumerable

Following the above intuition about characteristic vectors and the diagonal, we shall now prove formally a fundamental result about Turing machines: there is no Turing machine that accepts the language L_d .

Theorem 9.2: L_d is not a recursively enumerable language. That is, there is no Turing machine that accepts L_d .

PROOF: Suppose L_d were $L(M)$ for some TM M . Since L_d is a language over alphabet $\{0, 1\}$, M would be in the list of Turing machines we have constructed, since it includes all TM's with input alphabet $\{0, 1\}$. Thus, there is at least one code for M , say i ; that is, $M = M_i$.

Now, ask if w_i is in L_d .

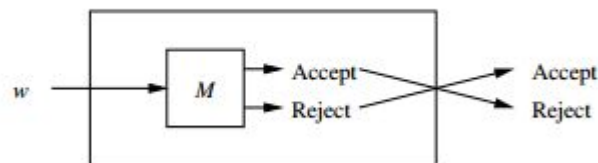
- If w_i is in L_d , then M_i accepts w_i . But then, by definition of L_d , w_i is not in L_d , because L_d contains only those w_j such that M_j does *not* accept w_j .
- Similarly, if w_i is not in L_d , then M_i does not accept w_i . Thus, by definition of L_d , w_i is in L_d .

Theorem 9.3: If L is a recursive language, so is $\overline{L}$.

PROOF: Let $L = L(M)$ for some TM M that always halts. We construct a TM $\overline{M}$ such that $\overline{L} = L(\overline{M})$ by the construction suggested in Fig. 9.3. That is, $\overline{M}$ behaves just like M . However, M is modified as follows to create $\overline{M}$:

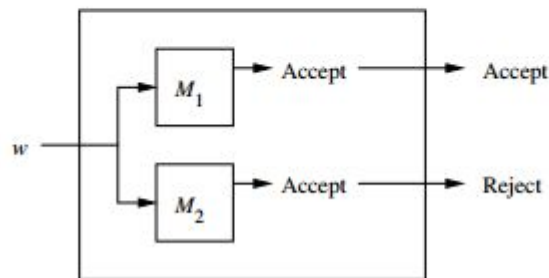
1. The accepting states of M are made nonaccepting states of $\overline{M}$ with no transitions; i.e., in these states $\overline{M}$ will halt without accepting.
2. $\overline{M}$ has a new accepting state r ; there are no transitions from r .
3. For each combination of a nonaccepting state of M and a tape symbol of M such that M has no transition (i.e., M halts without accepting), add a transition to the accepting state r .

Since M is guaranteed to halt, we know that $\overline{M}$ is also guaranteed to halt. Moreover, $\overline{M}$ accepts exactly those strings that M does not accept. Thus $\overline{M}$ accepts $\overline{L}$. $\square$



Theorem 9.4: If both a language L and its complement are RE, then L is recursive. Note that then by Theorem 9.3, $\overline{L}$ is recursive as well.

PROOF: The proof is suggested by Fig. 9.4. Let $L = L(M_1)$ and $\overline{L} = L(M_2)$. Both M_1 and M_2 are simulated in parallel by a TM M . We can make M a two-tape TM, and then convert it to a one-tape TM, to make the simulation easy and obvious. One tape of M simulates the tape of M_1 , while the other tape of M simulates the tape of M_2 . The states of M_1 and M_2 are each components of the state of M .



. What about L_d complement?

What are possibilities? Why is it RE but not REC?

Q: What about $L_u = \{ \langle M, w \rangle \mid w \in L(M) \}$ and its complement?

Q: Prove that L_u is RE and its complement is non-RE?

Q: Why TM cant solve all the problems?

.Reductions

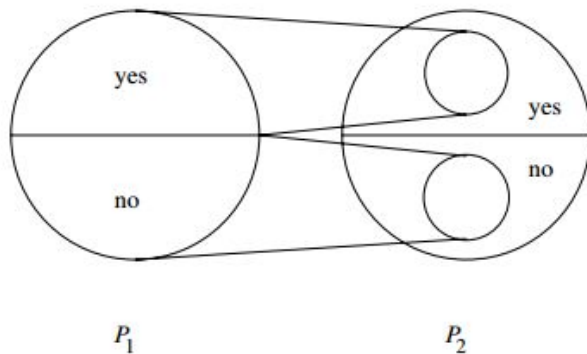
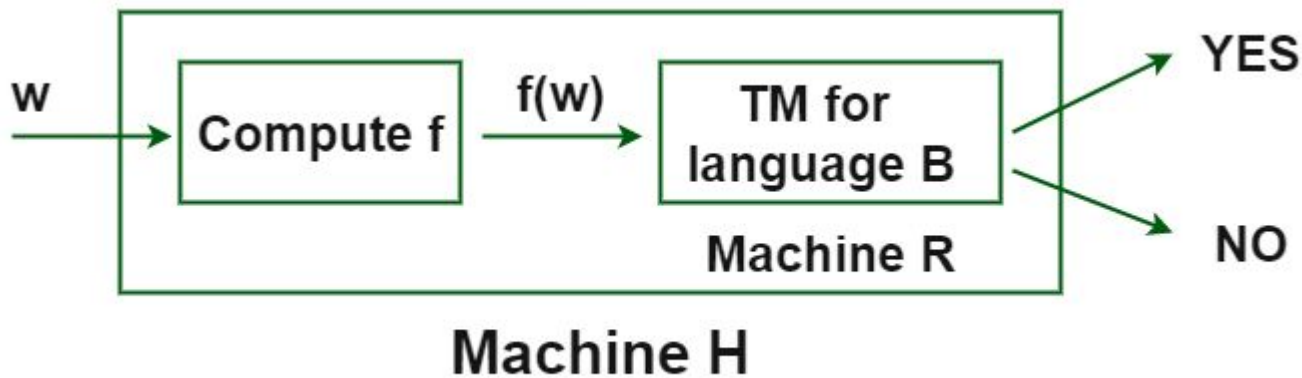
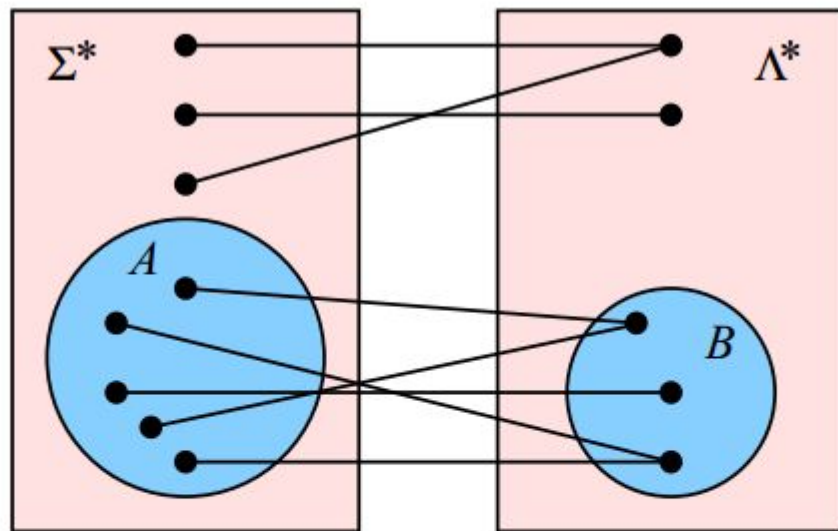


Figure 9.7: Reductions turn positive instances into positive, and negative to negative



H accepts w if R accepts $f(w)$ if $f(w) \in B$ if $w \in A$



- σ is a reduction from A to B .
- Notation: $A \leq_m B$ (many-to-one reduction) or $A \leq_{TM} B$ (Turing reduction).
- The membership problem for A is no more difficult than the membership problem for B .
- Example: $HP \leq_m MP$ and $MP \leq_m HP$.

Theorem: Let A, B be languages along with a reduction $A \leq_m B$.
If B is recursive, then A is also recursive.
Contrapositively, if A is not recursive, then B is also not recursive.

Proof

- Let B be recursive.
- Let σ be the reduction map $A \leq_m B$.
- Since B is r.e., A is r.e. too (by the previous theorem).
- σ is also a reduction map for $\bar{A} \leq_m \bar{B}$.
- $\bar{B}$ is recursive and so r.e.
- By the previous theorem, $\bar{A}$ is r.e. too.
- Since A and $\bar{A}$ are both r.e., A is recursive.

Properties of Reduction :

- If $A \leq B$, and A is undecidable then B is also undecidable.
- If $A \leq B$, and B is undecidable then A need not to be undecidable.
- If $A \leq B$, and A is decidable then B need not to be undecidable.
- If $A \leq B$, and B is decidable then A is also decidable.
- If $A \leq B$, and B is recursive then A is also recursive.
- If $A \leq B$, and A is recursive then B need not to be recursive.
- If $A \leq B$, and B is recursive enumerable then A is also recursive enumerable.
- If $A \leq B$, and A is recursive enumerable then B need not to be recursive enumerable.
- If $A \leq B$, and B is P-problem then A is also P-problem.
- If $A \leq B$, and A is P-problem then B need not to be P-problem.
- If $A \leq B$, and B is NP-problem then A is also NP-problem.
- If $A \leq B$, and A is P-problem then B need not to be P-problem.
- If $A \leq B$ and $B \leq P$ then $A \leq P$ (transitivity).
- If $A \leq B$ and $B \leq A$ then A and B are polynomial equivalent.
- If $A \leq B$ and A is not REL then B is also not REL.
- If $A \leq B$, and A is not P-problem then B is also not P-problem.
- If $A \leq B$ and A is not recursive problem then B is also not recursive problem.

Proposition: Let w be a fixed string over Σ . The problem whether a given Turing machine M accepts w is undecidable.

Proposition: The problem whether a given Turing machine M accepts any string at all is undecidable.

Proposition: The problem whether a given Turing machine M accepts all the strings over Σ is undecidable.

Proposition: The problem whether a given Turing machine M accepts only finitely many strings is undecidable.

.Rice's Theorem is a fundamental result in **computability theory** and **automata theory** that states:

Every non-trivial semantic property of a Turing machine is undecidable.

Key Points:

1. **Semantic Property:** A property that depends on the *language recognized* by a Turing machine (TM), not on its syntactic structure (e.g., the specific implementation of the TM).
 - Example: "Does the TM accept all strings?" (semantic)
 - Non-example: "Does the TM have more than 5 states?" (syntactic, decidable).
2. **Non-trivial Property:** A property that is neither always true nor always false for all TMs.
 - Example: "Does the TM accept the empty string?" (non-trivial)
 - Trivial example: "Does the TM accept some input?" (always true, trivial).

Implications:

- Rice's Theorem shows that **no algorithm can decide** whether a given Turing machine's language has a particular non-trivial property.
- Examples of undecidable problems (by Rice's Theorem):
 - Does the TM accept a finite language?
 - Does the TM accept a regular language?

Why is it Important?

- It generalizes many undecidability results in computability theory.
- It highlights the limitations of algorithmic analysis for program behavior (e.g., verifying correctness, halting, etc.).

Proof Sketch:

1. Assume (for contradiction) that a property P is decidable.
2. Construct a TM that reduces the Halting Problem to P
3. Since the Halting Problem is undecidable,
4. P must also be undecidable.

Example Application:

- **Question:** "Does a given TM accept exactly the strings with an even number of 1s?"
- **Rice's Theorem says:** This is undecidable because it's a non-trivial semantic property.

Limitations:

- Rice's Theorem applies only to **RE languages**.
- It does not apply to weaker models (e.g., DFAs, PDAs), where many properties *are* decidable.

Q. Is given RE finite?

Q. $L = \{ \langle M \rangle \mid 010 \text{ is in } L(M) \}$ or 010 belong to given RE language

Q. Does given TM accept regular language $\Rightarrow$ Is RE is regular?

Q. Does given TM accept empty language?

Q. Does given type-0 Grammar generate regular language

Q. Does TM produce an output

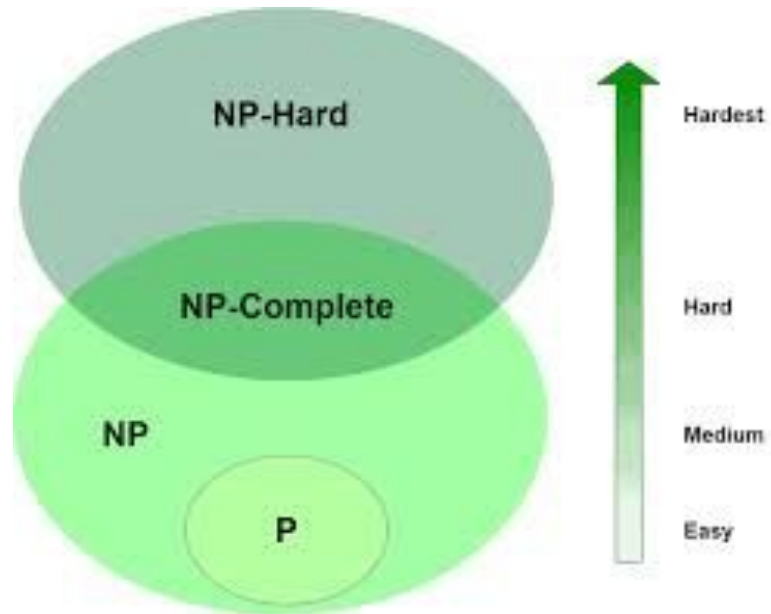
Q. Does given TM has 10 states?

Q. $L = \{ \langle M1, M2 \rangle \mid L(M1) = L(M2) \}$

Q. Does the given TM halt?

Q. Is given RE language accepted by TM

Complexity Theory



P Class

- Let $T: \mathbb{N} \rightarrow \mathbb{N}$ be some function.
- **Definition:** A language L is in $\text{DTIME}(T(n))$ if there's a TM that decides L in time $O(T(n))$.
- **Defintion:** Class $P = \bigcup \text{DTIME}(n^c)$.
- **Definition.** A TM M is a *polynimial time* TM if there's a polynomial function $q: \mathbb{N} \rightarrow \mathbb{N}$ such that for every input $x \in \{0,1\}^*$, M halts within $q(|x|)$ steps.

Polynomial function. $q(n) = n^c$ for some constant c

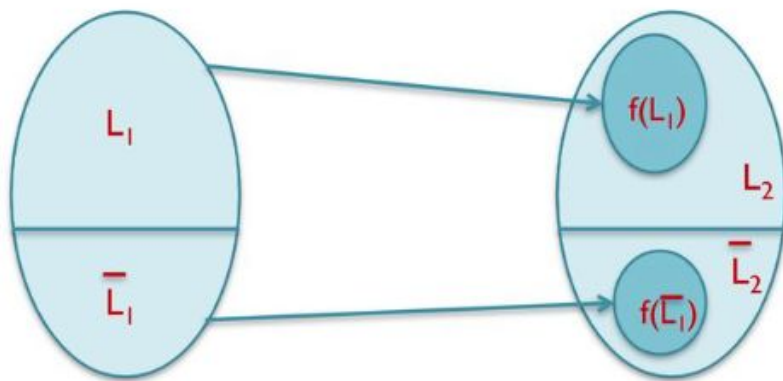
- **Definition.** A language $L \subseteq \{0,1\}^*$ is in **NP** if there's a polynomial function $p: \mathbb{N} \rightarrow \mathbb{N}$ and a polynomial time TM M (called the verifier) such that for every x ,

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)} \text{ s.t. } M(x, u) = 1$$

Polynomial time reduction

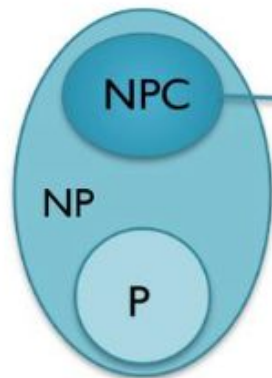
- **Definition.** We say a language $L_1 \subseteq \{0,1\}^*$ is polynomial time (Karp) reducible to a language $L_2 \subseteq \{0,1\}^*$ if there's a polynomial time computable function f s.t.

$$x \in L_1 \iff f(x) \in L_2$$



NP-completeness

- **Definition.** A language L' is *NP-hard* if for every L in NP , $L \leq_p L'$. Further, L' is *NP-complete* if L' is in NP and is NP-hard.
- **Observe.** If L' is NP-hard and L' is in P then $P = NP$. If L' is NP-complete then L' in P if and only if $P = NP$.



Hardest problems inside NP in the sense that if one NPC problem is in P then all problems in NP is in P.

A natural NP-complete problem

- **Definition.** A boolean formula is in Conjunctive Normal Form (CNF) if it is an AND of OR of literals.

e.g. $\varphi = (x_1 \vee x_2) \wedge (x_3 \vee \neg x_2)$

- **Definition.** Let **SAT** be the language consisting of all *satisfiable CNF formulae*.

- **Theorem.** (Cook-Levin) **SAT** is NP-complete.

Easy to see that **SAT** is in **NP**.

Need to show that **SAT** is NP-hard.